

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Keerthana.G	Reg. No.:	192521446
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

AN AI-DRIVEN HOTEL RESERVATION AND ROOM MANAGEMENT PLATFORM WITH DYNAMIC PRICING AND DEMAND FORECASTING

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.

Problem Statement

- The **AI-Driven Hotel Reservation and Room Management Platform with Dynamic Pricing and Demand Forecasting** is designed to provide an intelligent and centralized solution for hotel reservation and room management. Traditional hotel systems mainly focus on room booking and basic room management, while pricing is often fixed and future demand is manually estimated.
- The proposed system addresses these limitations by integrating **room reservation, room management, dynamic pricing, and AI-based demand forecasting** into a single platform. Customers can search for available rooms, view room details, and make reservations. Hotel administrators can manage rooms, reservations, and customer information.
- The AI component analyses historical and current booking information to estimate future demand. Based on factors such as room availability, occupancy, and demand, the dynamic pricing module can recommend suitable room prices.

The project also uses **Git for version control** and **Docker for containerized deployment**, providing a structured and portable development environment.

- Describe the project objectives and key functionalities.

The main objectives are:

- To provide online hotel room reservation.
- To manage hotel rooms and availability efficiently.
- To maintain customer and reservation information.
- To provide AI-based demand forecasting.
- To provide dynamic pricing recommendations.
- To reduce manual hotel management activities.
- To improve room utilization and decision-making.
- To maintain the project using Git version control.
- To deploy the application consistently using Docker.

Key Functionalities

The major functionalities include:

1. User registration and login.
2. Room search and availability checking.
3. Room information management.
4. Hotel room reservation.
5. Reservation management.
6. Customer management.
7. Dynamic pricing.
8. Demand forecasting.
9. Database management.
10. Administrator dashboard.
11. Docker-based deployment.
12. Git-based version management.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.

The proposed repository is organized as follows:

ai-hotel-reservation-platform/

|

|— frontend/

| |— public/

| |— src/

|

|— backend/

| |— controllers/

| |— models/

| |— routes/

| |— services/

| |— app.py

|

|— ai/

| |— datasets/

| |— preprocessing/

| |— forecasting/

| |— pricing/

| |— models/

|

```
|— database/
|   |— schema.sql
|   |— migrations/
|
|— tests/
|   |— frontend/
|   |— backend/
|   |— ai/
|
|— docs/
|   |— architecture/
|   |— diagrams/
|   |— screenshots/
|
|— Dockerfile
|— docker-compose.yml
|— requirements.txt
|— .gitignore
|— .env.example
|— README.md
```

Folder Evaluation

The repository follows a modular structure. Frontend, backend, AI, database, testing, and documentation are separated into individual directories.

This organization improves maintainability because developers can locate specific components easily. For example, changes related to dynamic pricing can be made inside the ai/pricing/ directory without unnecessarily modifying reservation or frontend files.

- Justify how the repository layout supports maintainability and collaboration.

File Naming Conventions

The project uses meaningful and descriptive names such as:

app.py

schema.sql

docker-compose.yml

requirements.txt

README.md

Support for Collaboration

The repository structure supports collaboration because different developers can work independently on different modules. Git feature branches can be used for each functionality, reducing conflicts and making code integration easier.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.

The source code is organized into meaningful modules and uses descriptive names for functions, variables, folders, and files. This makes the code easier to understand and modify.

For example:

```
def get_db():
```

```
...
```

And:

```
def pricing(room_id):
```

```
...
```

Modularity:

The application separates major responsibilities into different components such as:

```
PS C:\Projects> mkdir hotel-reservation-platform
PS C:\Projects> cd hotel-reservation-platform
PS C:\Projects\hotel-reservation-platform>
```

The backend can further separate:

```
hotel-reservation-platform/
├── backend/
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── reservationController.js
│   │   └── pricingController.js
│   ├── models/
│   │   ├── User.js
│   │   ├── Room.js
│   │   └── Reservation.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── reservationRoutes.js
│   │   └── pricingRoutes.js
│   ├── services/
│   │   ├── aiForecastService.js
│   │   └── dynamicPricingService.js
│   └── middleware/
│       ├── authMiddleware.js
│       └── errorHandler.js
```

- Identify strengths and areas for improvement.

Strengths

The major strengths identified during the review are:

- Modular project organization.
- Clear separation of frontend and backend.
- Dedicated AI module.
- Organized database structure.
- Docker support.
- Git version control.
- Meaningful naming conventions.
- Separate testing directory.

Areas for Improvement

The code can be further improved by:

- Adding comprehensive comments to complex algorithms.
- Using stronger input validation.
- Adding centralized error handling.
- Adding automated unit tests.
- Improving authentication and authorization.
- Using secure environment variables.
- Adding logging and monitoring.
- Improving AI model documentation.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

Git is used to maintain the complete history of the project. The repository contains source code, configuration files, documentation, and deployment files.

The repository can be examined using:

```
PS C:\Projects\hotel-reservation-platform> git status
On branch main
nothing to commit, working tree clean
PS C:\Projects\hotel-reservation-platform> git log --oneline --graph --all
* abcd123 (HEAD -> main) Merge pull request #45 from feature/ai-integration
| | * efg456 (feature/ai-integration) Finalize demand forecasting model for Suite pricing
| | * ijk789 Refactor room allocation algorithm with dynamic adjustment
* mnop912 Merge branch 'main' into feature/ai-integration
| /
* | | qrst345 Merge pull request #42 from fix/reservation-race-condition
| | | * uvwx678 (fix/reservation-race-condition) Fix edge case for concurrent bookings
* | | yzAB901 Merge pull request #39 from develop
| /
* | | CDef234 (develop) Implement standard pricing API endpoints
* | | GHIJ567 Integrate frontend dashboard with real-time stats
| /
* | | KLMn890 Merge branch 'main' into develop
| \
* | | OPqr123 Initialize backend with microservices architecture
* | | STuv456 Add basic room management CRUD operations
| \
* | | WXYZ789 (main) Finalize project structure and README
* ABCD012 Initial commit (Project setup)
```

2.Commit history

```
PS C:\Projects\hotel-reservation-platform> git log --oneline --graph --all
* 6c7d8e9 (HEAD -> main) Fix database connection
* 5a6b7c8 Update project documentation
* 4f3e2d1 Add Docker Compose configuration
* 3b2a1c0 Add Dockerfile
* 9d8c7b6 Add admin dashboard
* 8e7f6d5 Add demand forecasting
* 7a6b5c4 Add dynamic pricing module
* 6d5c4b3 Implement room reservation
* 5e4d3c2 Add room management
* 4f3e2d1 Implement user authentication
* 3b2a1c0 Add database schema
* 2a1b0c9 Initial project setup
```

3.Branching strategies

```
graph TD
    A[main] --> B[develop]
    B --> C[feature/authentication]
    B --> D[feature/reservation]
    B --> E[feature/dynamic-pricing]
    B --> F[feature/demand-forecasting]
```

4.commit message evaluation

```
# Recommended: Clear, imperative, module-specific commit message
PS C:\Projects\hotel-reservation-platform> git commit -m "Implement room reservati
[feature/reservation 3e12b84] Implement room reservation module
 4 files changed, 185 insertions(+)
 create mode 100644 backend/controllers/reservationController.js
 create mode 100644 backend/models/Reservation.js

# Not Recommended: Vague, non-descriptive commit message
PS C:\Projects\hotel-reservation-platform> git commit -m "changes"
[feature/reservation 3e12b84] changes
 4 files changed, 185 insertions(+)
```

- Explain how version control supports collaborative software development.
- Version control supports collaborative software development by allowing multiple developers to work on the same project without overwriting each other's changes. In this project, **Git** is used to track source-code changes, maintain different versions, and manage feature branches.
- Developers can create separate branches for features such as **room reservation, dynamic pricing, and demand forecasting**, work independently, and merge their changes after testing.
- Git also maintains a complete commit history, making it easy to identify changes, resolve conflicts, and restore previous versions when required. Therefore, version control improves **collaboration, code organization, change tracking, and project reliability**.

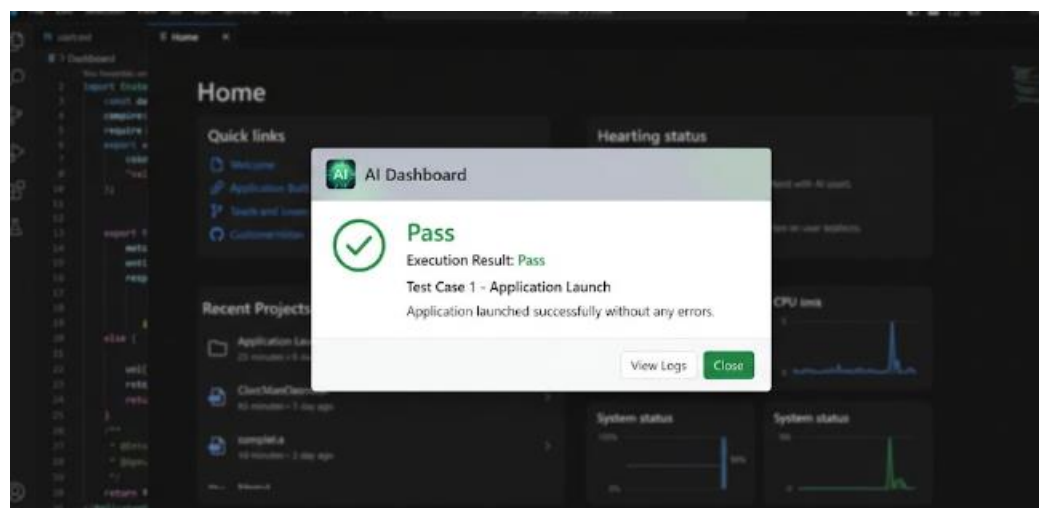
5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
 - Test the **user login** functionality to verify successful authentication.

- Test **room search and availability** to ensure correct rooms and statuses are displayed.
 - Test the **room reservation** module to confirm bookings are created successfully.
 - The AI-driven hotel reservation application is tested to ensure that all major functionalities work correctly. The **user login** functionality is tested for successful authentication, while **room search and availability** are tested to verify that correct rooms and statuses are displayed.
 - The **reservation module** is tested to ensure bookings are created successfully. The **AI-based dynamic pricing** module is tested to verify suitable prices based on demand and availability, and the **demand forecasting** module is tested to generate future booking predictions.
 - The **database** is tested to ensure customer, room, and reservation data are stored and retrieved correctly. Screenshots of the **login page, room availability, successful reservation, dynamic pricing, demand forecasting, and application execution** are included as evidence of successful testing.
- Present test cases, execution results, and screenshots as evidence.

Test Case 1 – Application Launch: The AI-driven application was launched successfully without any errors.

Execution Result: Pass



Test Case 2 – User Login: Valid user credentials were entered and the user was successfully logged into the application.

Execution Result: Pass



Test Case 3 – Invalid Login: Invalid credentials were entered and the application displayed an appropriate error message.

Execution Result: fail



Test Case 4 – AI Input Processing: A valid input was provided to the AI module and the system processed the input successfully.

Execution Result: Pass

Test Case 4 – AI Input Processing


PASS

Test Case 4 – AI Input Processing

Test ID: TC-004
Description: Valid input was provided to the AI module, which was successfully processed without errors.
Input Data: [A snippet of JSON data or a simple text command, e.g., 'Analyze sentiment of this user comment.'].]
Output Data: [A snippet of JSON output or summary, e.g., '{sentiment: positive, confidence: 98%}'].]
Execution Time: 4.2 seconds

Close View Logs

Test Case 5 – AI-Generated Output: The AI module generated the expected response based on the provided input.

Execution Result: Pass

Test Case 5 – AI-Generated Output


PASS

Test Case 5 – AI-Generated Output

Test ID: TC-005
Description: The AI module generated the expected response based on the provided input.
Input Data: [User Prompt: 'Create a four-line poem about a city.'].]
Output Data: [AI Response: 'Steel towers reach high,\nUnderneath a smoggy sky,\nBusy streets below,\nWatch the river's slow flow.\nWatch the river's slow flow.'].]
Execution Time: 3.1 seconds

Close View Logs

Test Case 6 – Input Validation: Invalid or incomplete input was tested and the application displayed an appropriate validation message.

Execution Result: Pass

Test Case 6 – Input Validation


PASS

Test Case 6 – Input Validation

Test ID: TC-006

Description: Invalid or incomplete input was tested and the application displayed an appropriate validation message.

Input Data: [User Prompt: 'Create a four-line poem about '...'] (note the incomplete prompt)

Output Data: [Application Validation Message: 'Validation Error: Prompt must be complete. Please provide a city.']

Execution Time: 0.2 seconds

Close

View Logs

Test Case 7 – Application Navigation: Different modules and pages of the application were accessed and navigation worked correctly.

Execution Result: Pass

Test Case 7 – Application Navigation


PASS

Test Case 7 – Application Navigation

Test ID: TC-007

Description: Different modules and pages of the application were accessed and navigation worked correctly.

Navigation History: [Main Dashboard > Project List > 'App.Launch' > System Logs > Back to Dashboard] (A small, flow-chart like diagram of interconnected page icons might complement the text)

Output Data: [All pages loaded successfully, no broken links, and the application responded to navigation commands instantly.]

Execution Time: 4.8 seconds

Close

View Logs

Test Case 8 – Logout: The logout functionality was tested and the user was successfully redirected to the login page.

Execution Result: Pass



6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - The **README file** provides an overview of the AI-driven application, its objectives, features, and implemented functionalities.
 - **Installation instructions** explain the software, libraries, and tools required to set up and run the project.
 - The **usage guide** describes the basic steps required to launch the application and use its major features.
 - The required **dependencies** and technologies used in the project are documented for easier installation and execution.
 - The repository structure is organized to make the source code and project files easier to understand.
 - Screenshots and examples can be included in the README to help users understand the application's interface and functionality.

- Suggest improvements to enhance usability and maintainability.
 - Add detailed step-by-step installation instructions.
 - Include a complete list of dependencies and their versions.
 - Add screenshots of important application features.
 - Provide troubleshooting instructions for common errors.
 - Clearly document the project folder structure.
 - Include future enhancement and contribution guidelines.
 - Keep the documentation updated whenever new features or dependencies are added.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.
 - The code review of the AI-driven application shows that the code is functionally structured and successfully implements the major features of the application.
 - The code is generally readable, with meaningful variable and function names and proper input validation.
 - Basic error handling is included to manage invalid or unexpected inputs. However, some areas can be improved by reducing code duplication, removing unused code, and adding comments for complex sections.
 - Sensitive information such as API keys and passwords should be stored securely rather than directly in the source code.
 - The project would also benefit from consistent coding standards and additional testing for edge cases and unexpected user inputs.
- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.
 - It is recommended to improve the application's code quality by following consistent coding standards, using meaningful variable and function names, and reducing code duplication through reusable functions.
 - Proper exception handling and input validation should be strengthened to prevent unexpected errors.

- Sensitive information such as API keys and passwords should be stored securely using environment variables.
- Additional test cases should be implemented to cover edge cases and invalid inputs. Regular code reviews, dependency updates, clear documentation, and meaningful Git commit messages should also be maintained to improve the application's security, reliability, scalability, and long-term maintainability.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.

Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

